

CONCEPTS OF PROGRAMMING LANGUAGES

Chapter 3

Describing Syntax and Semantics



ROBERT W. SEBESTA

12/E

ISBN 0-321-49362-1

Chapter 3 Topics

- Introduction
- The General Problem of Describing Syntax
- Formal Methods of Describing Syntax
- Attribute Grammars
- Describing the Meanings of Programs:
Dynamic Semantics

Introduction

- **Syntax**: the form or structure of the expressions, statements, and program units
- **Semantics**: the meaning of the expressions, statements, and program units
- **Syntax** and **semantics** provide a **language's definition**
 - Users of a language definition
 - Other language designers
 - Implementers
 - Programmers (the users of the language)

The General Problem of Describing Syntax: Terminology

- A *sentence* is a string of characters over some alphabet
- A *language* is a set of sentences
- A *lexeme* is the lowest level syntactic unit of a language (e.g., *, sum, begin)
- A *token* is a category of lexemes (e.g., identifier)

-
- Lexemes are the lowest-level syntactic units.
 - The description of lexemes can be given by a lexical specification.

Java Lexical Structure

- <https://docs.oracle.com/javase/specs/jls/se8/html/jls-3.html>
- <https://home.unicode.org/>

Java Lexical Translation

- A raw Unicode character stream is translated into a sequence of tokens, using the following three lexical translation steps, which are applied in turn:
 - A translation of Unicode escapes ([§3.3](#)) in the raw stream of Unicode characters to the corresponding Unicode character.
 - A Unicode escape of the form `\uxxxx`, where `xxxx` is a hexadecimal value, represents the UTF-16 code unit whose encoding is `xxxx`. This translation step allows any program to be expressed using only ASCII characters.

-
- A translation of the Unicode stream resulting from step 1 into a stream of input characters and line terminators ([§3.4](#)).
 - A translation of the stream of input characters and line terminators resulting from step 2 into a sequence of input elements ([§3.5](#)) which, after white space ([§3.6](#)) and comments ([§3.7](#)) are discarded, comprise the tokens ([§3.5](#)) that are the terminal symbols of the syntactic grammar ([§2.3](#)).

C# Lexical Structures

- <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/lexical-structure>

Python

- The lexical structure of a programming language is the set of basic rules that govern how you write programs in that language.

-
- It is the lowest-level syntax of the language and specifies such things as what variable names look like and which characters denote comments. Each Python source file, like any other text file, is a sequence of characters. You can also usefully consider it as a sequence of lines, tokens, or statements.

-
- These different lexical views complement and reinforce each other. Python is very particular about program layout, especially with regard to lines and indentation, so you'll want to pay attention to this information if you are coming to Python from another language.

Formal Definition of Languages

- **Recognizers**

- A recognition device reads input strings over the alphabet of the language and decides whether the input strings belong to the language
- Example: syntax analysis part of a compiler
 - Detailed discussion of syntax analysis appears in Chapter 4

- **Generators**

- A device that generates sentences of a language
- One can determine if the syntax of a particular sentence is syntactically correct by comparing it to the structure of the generator

-
- A Python program is composed of a sequence of *logical lines*, each made up of one or more *physical lines*. Each physical line may end with a comment. A hash sign (#) that is not inside a string literal begins a comment. All characters after the # and up to the physical line end are part of the comment, and the Python interpreter ignores them.

-
- A line containing only whitespace, possibly with a comment, is known as a *blank line*, and Python totally ignores it. In an interactive interpreter session, you must enter an empty physical line (without any whitespace or comment) to terminate a multiline statement.

-
- In Python, the end of a physical line marks the end of most statements. Unlike in other languages, you don't normally terminate Python statements with a delimiter, ...

Python Lexical Categories

Table 2.1: Python's Lexical Categories

Identifier	Names that the programmer defines
Operators	Symbols that operate on data and produce results
Delimiters	Grouping, punctuation, and assignment/binding symbols
Literals	Values classified by types: e.g., numbers, truth values, text
Comments	Documentation for programmers reading code

BNF and Context-Free Grammars

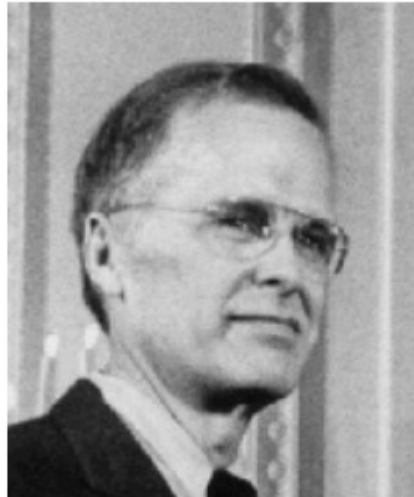
- Context-Free Grammars
 - Developed by Noam Chomsky in the mid-1950s
 - Language generators, meant to describe the syntax of natural languages
 - Define a class of languages called context-free languages
- Backus-Naur Form (1959)
 - Invented by John Backus to describe the syntax of Algol 58
 - BNF is equivalent to context-free grammars

What is BNF?

- It stands for Backus-Naur Form
- It is a formal, mathematical way to specify context-free grammars
- It is precise and unambiguous
- Before BNF, people specified programming languages ambiguously, i.e., with English

- John Backus presented a new notation containing most of the elements of BNF at a UNESCO conference
- His presentation was about Algol 58
- Peter Naur read this report and found that he and Backus interpreted Algol differently
- He wanted even more precision
- So he created what we now know as BNF for Algol 60
- Thus BNF was first published in *Algol 60 Report*

Who was John Backus?



- Backus invented FORTRAN (“FORMula TRANslator”), the first high-level language *ever*, circa 1954
- Major influence on the invention of functional programming in 1970’s
- Won the 1977 Turing Award for BNF and FORTRAN

Who was Peter Naur?



- Danish astronomer turned computer scientist
- Born in 1928; picture on left is from 1968

BNF Fundamentals

- In BNF, abstractions are used to represent classes of syntactic structures—they act like syntactic variables (also called *nonterminal symbols*, or just *terminals*)
- *Terminals* are lexemes or tokens
- A rule has a left-hand side (LHS), which is a nonterminal, and a right-hand side (RHS), which is a string of terminals and/or nonterminals

What does BNF look like?

- Like this:

`<number> ::= <digit> | <number> <digit>`

`<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`

- “::=” means “is defined as” (some variants use “:=” instead)
- “|” means “or”
- Angle brackets mean a nonterminal
- Symbols without angle brackets are terminals

More BNF Examples

- `<while loop> ::= while (<condition>)
 <statement>`
- `<assignment statement> ::= <variable> =
 <expression>`
- `<statement list> ::= <statement> |
 <statement list> <statement>`
- `<unsigned integer> ::= <digit> |
 <unsigned integer><digit>`

BNF for Expressions

`<expression> ::= <expression> + <term>`
`| <expression> - <term>`
`| <term>`

`<term> ::= <term> * <factor>`
`| <term> / <factor>`
`| <factor>`

`<factor> ::= <primary> ^ <factor>`
`| <primary>`

`<primary> ::= <primary>`
`| <element>`

`<element> ::= ( <expression> )`
`| <variable>`
`| <number>`

BNF Fundamentals (continued)

- Nonterminals are often enclosed in angle brackets
 - Examples of BNF rules:
`<ident_list> → identifier | identifier, <ident_list>`
`<if_stmt> → if <logic_expr> then <stmt>`
- Grammar: a finite non-empty set of rules
- A *start symbol* is a special element of the nonterminals of a grammar

BNF Rules

- An abstraction (or nonterminal symbol) can have more than one RHS

```
<stmt> → <single_stmt>  
        | begin <stmt_list> end
```

What is EBNF?

- EBNF is a few simple extensions to BNF which make expressing grammars more convenient; adds “syntactic sugar”
- Thus it is more concise than BNF
- EBNF is no more “powerful” than BNF; that is, anything that can be expressed in EBNF can also be expressed in BNF
- EBNF is widely used as the de facto standard to define programming languages

What are the Extensions?

- They vary, but often are derived from regular expression syntax
- “*” (The Kleene Star): means 0 or more occurrences
- “+” (The Kleene Cross): means 1 or more occurrences
- “?”: means 0 or 1 occurrences (sometimes “[...]” used instead)
- Use of parentheses for grouping

BNF vs EBNF

- Grammar for decimal numbers in plain BNF:

`<expr> ::= '-' <num> | <num>`

`<num> ::= <digits>`

`| <digits> '.' <digits>`

`<digits> ::= <digit>`

`| <digit> <digits>`

`<digit> ::= '0' | '1' | '2' | '3' |`

`'4' | '5' | '6' | '7' | '8' | '9'`

BNF vs EBNF

- Same grammar in EBNF:

```
<expr>  := '-'? <digit>+ ('.' <digit>+)?  
<digit> := '0' | '1' | '2' | '3' | '4' |  
          '5' | '6' | '7' | '8' | '9'
```

- So much more concise!
- An optional '-', one or more digits, an optional decimal point followed by one or more digits

Simple Conversions

- If you have a rule such as:

`<expr> ::= <digits>`

`<digits> ::= <digit>`

`| <digit> <digits>`

- You can replace it with:

`<expr> ::= <digit>+`

More Simple Conversions

- If you have a rule such as:

`<expr> ::= <digits> | empty`

- You can replace it with:

`<expr> ::= <digit>*`

More Simple Conversions

- If you have a rule such as:

```
<id> ::= <letter>  
        | <id><letter>  
        | <id><digit>
```

- You can replace it with:

```
<id> ::= <letter> (<letter> | <digit>)*
```

EBNF for Lisp

```
s_expression ::= atomic_symbol  
               | "(" s_expression "." s_expression ")"  
               | list  
list ::= "(" s_expression* ")"  
atomic_symbol ::= letter atom_part  
atom_part ::= empty  
              | letter atom_part  
              | number atom_part  
letter ::= "a" | "b" | " ..." | "z"  
number ::= "1" | "2" | " ..." | "9"
```

Formalizing Description – EBNF

- An EBNF description is an unordered list of EBNF rules.
- Each EBNF rule has three parts:
 - a left-hand side (LHS),
 - LHS is one italicized word (possibly with underscores) written in lower-case; it names the EBNF rule.
 - a right-hand side (RHS),
 - The RHS supplies a description of this name
 - It can include combinations of the four control forms
 - The $\Leftarrow$ character separating these two sides

Table 1.1: Control Forms of Right-Hand Sides

Sequence	Items appear left-to-right; their order is important.
Choice	Alternative items are separated by a (stroke); one item is chosen from this list of alternatives; their order is unimportant.
Option	The optional item is enclosed between [and] (square-brackets); the item can be either included or discarded.
Repetition	The repeatable item is enclosed between { and } (curly-braces); the item can be repeated zero or more times; yes, we can choose to repeat zero times, fact beginners often forget.

EBNF Special Characters

$\Leftarrow$, “is defined as”

|,

[,

],

{, and

}

An EBNF Description of Integers

The following EBNF rules describe how to write simple integers.¹ Their RHS illustrates every control form available in EBNF. An EBNF description: *integer*

EBNF Description: *integer*

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

integer $\Leftarrow$ [+|-]*digit*{*digit*}

We can paraphrase the meanings of these rules in English.

- A *digit* is defined as one of the ten alternative characters 0 through 9.
- An *integer* is defined as a sequence of three items: an optional sign (if it is included, it must be one of the alternatives + or -), followed by any *digit*, followed by a repetition of zero or more *digits*, where each repeated digit is independently chosen from the list of alternatives in the *digit* rule.

What do these EBNF rules mean? We can paraphrase their meaning in English

- The RHS of *integer* combines all the control forms in EBNF: sequence, option, choice, and repetition.

Equivalent EBNF Descriptions

EBNF Description: *integer* (equivalent, in 3 rules)

$sign \Leftarrow +|-$

$digit \Leftarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$integer \Leftarrow [sign]digit\{digit\}$

This EBNF description is not identical to the first, because it defines an extra *sign* rule that is then used in the *integer* rule. But these two EBNF descriptions are equivalent, because providing a named rule for $[+|-]$ does not change which symbols are legal. In fact, even if the names of all the rules are changed uniformly, exactly the same symbols are recognized as legal.

Extra names do not change the meanings of EBNF descriptions; nor do different names for the rules

EBNF Description: *z* (really *integer* with different rule names)

$x \Leftarrow +|-$

$y \Leftarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$z \Leftarrow [x]y\{y\}$

EBNF Description: *integer_set*

$integer_list \Leftarrow integer\{,integer\}$

$integer_set \Leftarrow \boxed{\{[integer_list]\}}$

Describing Lists

- Syntactic lists are described using recursion

$$\begin{aligned} \text{<ident_list>} &\rightarrow \text{ident} \\ &\quad | \text{ ident, <ident_list>} \end{aligned}$$

- A derivation is a repeated application of rules, starting with the start symbol and ending with a sentence (all terminal symbols)

An Example Grammar

$\langle \text{program} \rangle \rightarrow \langle \text{stmts} \rangle$

$\langle \text{stmts} \rangle \rightarrow \langle \text{stmt} \rangle \mid \langle \text{stmt} \rangle ; \langle \text{stmts} \rangle$

$\langle \text{stmt} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expr} \rangle$

$\langle \text{var} \rangle \rightarrow a \mid b \mid c \mid d$

$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle - \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{var} \rangle \mid \text{const}$

An Example Derivation

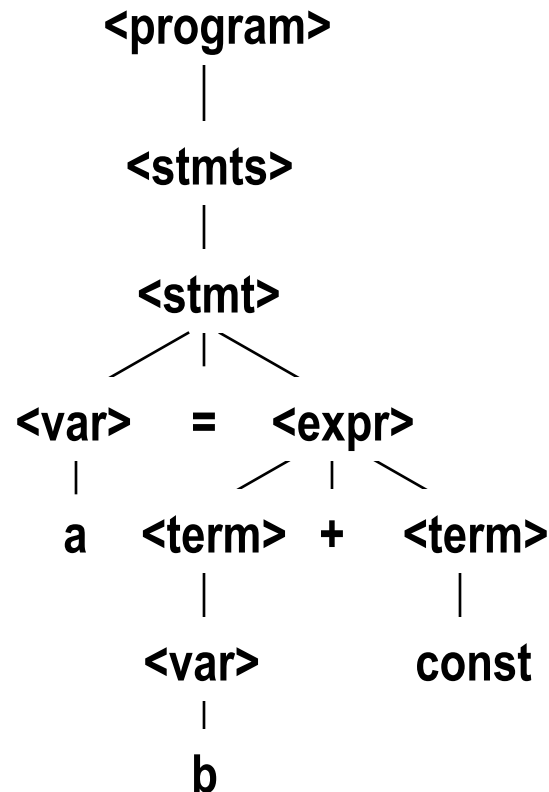
`<program> => <stmts> => <stmt>`
`=> <var> = <expr>`
`=> a = <expr>`
`=> a = <term> + <term>`
`=> a = <var> + <term>`
`=> a = b + <term>`
`=> a = b + const`

Derivations

- Every string of symbols in a derivation is a *sentential form*
- A *sentence* is a sentential form that has only terminal symbols
- A *leftmost derivation* is one in which the leftmost nonterminal in each sentential form is the one that is expanded
- A derivation may be neither leftmost nor rightmost

Parse Tree

- A hierarchical representation of a derivation



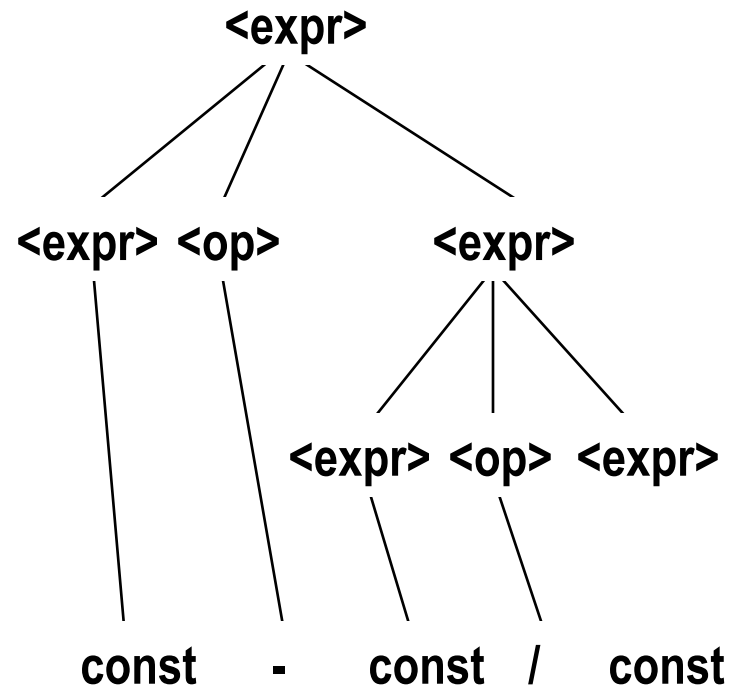
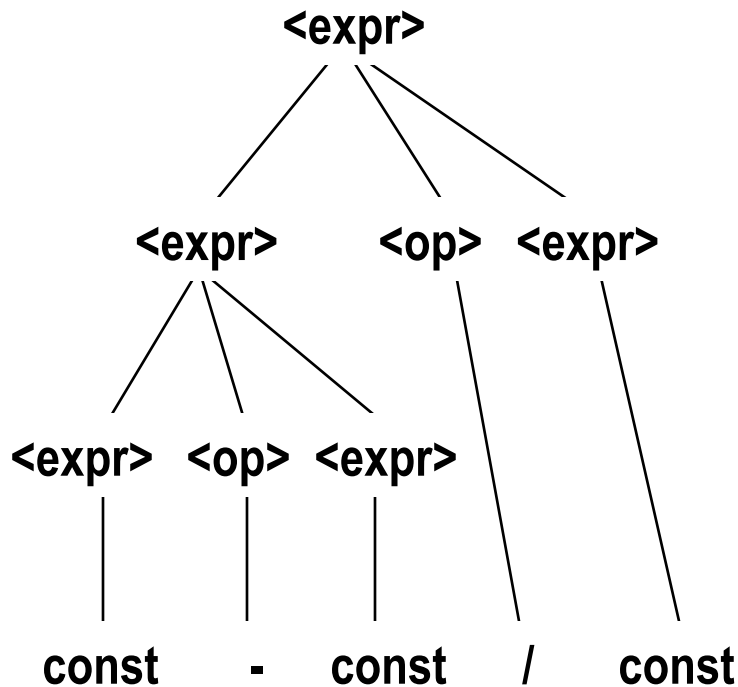
Ambiguity in Grammars

- A grammar is *ambiguous* if and only if it generates a sentential form that has two or more distinct parse trees

An Ambiguous Expression Grammar

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle \mid \text{const}$

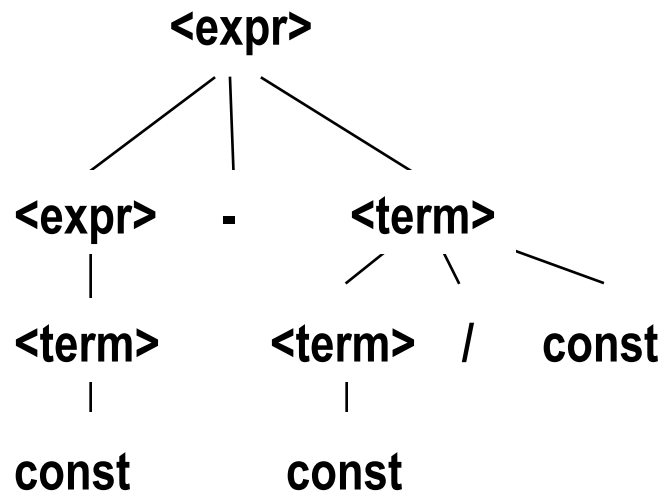
$\langle \text{op} \rangle \rightarrow / \mid -$



An Unambiguous Expression Grammar

- If we use the parse tree to indicate precedence levels of the operators, we cannot have ambiguity

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle - \langle \text{term} \rangle \mid \langle \text{term} \rangle$
 $\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle / \text{const} \mid \text{const}$

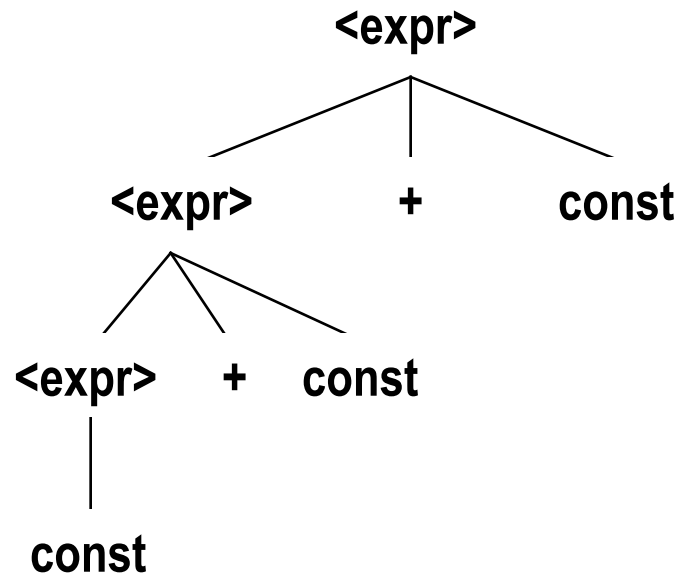


Associativity of Operators

- Operator associativity can also be indicated by a grammar

`<expr> -> <expr> + <expr> | const` (ambiguous)

`<expr> -> <expr> + const | const` (unambiguous)



Extended BNF

- Optional parts are placed in brackets []

`<proc_call> -> ident [ (<expr_list>) ]`

- Alternative parts of RHSs are placed inside parentheses and separated via vertical bars

`<term> → <term> (+|-) const`

- Repetitions (0 or more) are placed inside braces { }

`<ident> → letter {letter|digit}`

BNF and EBNF

- BNF

$$\begin{aligned}\langle \text{expr} \rangle &\rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \\ &| \langle \text{expr} \rangle - \langle \text{term} \rangle \\ &| \langle \text{term} \rangle\end{aligned}$$
$$\begin{aligned}\langle \text{term} \rangle &\rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \\ &| \langle \text{term} \rangle / \langle \text{factor} \rangle \\ &| \langle \text{factor} \rangle\end{aligned}$$

- EBNF

$$\begin{aligned}\langle \text{expr} \rangle &\rightarrow \langle \text{term} \rangle \{ (+ \mid -) \langle \text{term} \rangle \} \\ \langle \text{term} \rangle &\rightarrow \langle \text{factor} \rangle \{ (* \mid /) \langle \text{factor} \rangle \}\end{aligned}$$

Recent Variations in EBNF

- Alternative RHSs are put on separate lines
- Use of a colon instead of $\Rightarrow$
- Use of `opt` for optional parts
- Use of `oneof` for choices

Static Semantics

- Nothing to do with meaning
- Context-free grammars (CFGs) cannot describe all of the syntax of programming languages
- Categories of constructs that are trouble:
 - Context-free, but cumbersome (e.g., types of operands in expressions)
 - Non-context-free (e.g., variables must be declared before they are used)

Attribute Grammars

- Attribute grammars (AGs) have additions to CFGs to carry some semantic info on parse tree nodes
- Primary value of AGs:
 - Static semantics specification
 - Compiler design (static semantics checking)

Attribute Grammars : Definition

- **Def:** An attribute grammar is a context-free grammar $G = (S, N, T, P)$ with the following additions:
 - For each grammar symbol x there is a set $A(x)$ of attribute values
 - Each rule has a set of functions that define certain attributes of the nonterminals in the rule
 - Each rule has a (possibly empty) set of predicates to check for attribute consistency

Attribute Grammars: Definition

- Let $X_0 \rightarrow X_1 \dots X_n$ be a rule
- Functions of the form $S(X_0) = f(A(X_1), \dots, A(X_n))$ define *synthesized attributes*
- Functions of the form $I(X_j) = f(A(X_0), \dots, A(X_n))$, for $i \leq j \leq n$, define *inherited attributes*
- Initially, there are *intrinsic attributes* on the leaves

Attribute Grammars: An Example

- **Syntax**

`<assign> -> <var> = <expr>`

`<expr> -> <var> + <var> | <var>`

`<var> A | B | C`

- `actual_type`: **synthesized** for `<var>`
and `<expr>`
- `expected_type`: **inherited** for `<expr>`

Attribute Grammar (continued)

- **Syntax rule:** $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle[1] + \langle \text{var} \rangle[2]$

Semantic rules:

$\langle \text{expr} \rangle.\text{actual_type} \leftarrow \langle \text{var} \rangle[1].\text{actual_type}$

Predicate:

$\langle \text{var} \rangle[1].\text{actual_type} == \langle \text{var} \rangle[2].\text{actual_type}$

$\langle \text{expr} \rangle.\text{expected_type} == \langle \text{expr} \rangle.\text{actual_type}$

- **Syntax rule:** $\langle \text{var} \rangle \rightarrow \text{id}$

Semantic rule:

$\langle \text{var} \rangle.\text{actual_type} \leftarrow \text{lookup} (\langle \text{var} \rangle.\text{string})$

Attribute Grammars (continued)

- How are attribute values computed?
 - If all attributes were inherited, the tree could be decorated in top-down order.
 - If all attributes were synthesized, the tree could be decorated in bottom-up order.
 - In many cases, both kinds of attributes are used, and it is some combination of top-down and bottom-up that must be used.

Attribute Grammars (continued)

$\langle \text{expr} \rangle . \text{expected_type} \leftarrow \text{inherited from parent}$

$\langle \text{var} \rangle [1] . \text{actual_type} \leftarrow \text{lookup (A)}$

$\langle \text{var} \rangle [2] . \text{actual_type} \leftarrow \text{lookup (B)}$

$\langle \text{var} \rangle [1] . \text{actual_type} =? \langle \text{var} \rangle [2] . \text{actual_type}$

$\langle \text{expr} \rangle . \text{actual_type} \leftarrow \langle \text{var} \rangle [1] . \text{actual_type}$

$\langle \text{expr} \rangle . \text{actual_type} =? \langle \text{expr} \rangle . \text{expected_type}$

Semantics

- There is no single widely acceptable notation or formalism for describing semantics
- Several needs for a methodology and notation for semantics:
 - Programmers need to know what statements mean
 - Compiler writers must know exactly what language constructs do
 - Correctness proofs would be possible
 - Compiler generators would be possible
 - Designers could detect ambiguities and inconsistencies

Operational Semantics

- Operational Semantics
 - Describe the meaning of a program by executing its statements on a machine, either simulated or actual. The change in the state of the machine (memory, registers, etc.) defines the meaning of the statement
- To use operational semantics for a high-level language, a virtual machine is needed

Operational Semantics

- A *hardware* pure interpreter would be too expensive
- A *software* pure interpreter also has problems
 - The detailed characteristics of the particular computer would make actions difficult to understand
 - Such a semantic definition would be machine-dependent

Operational Semantics (continued)

- A better alternative: A complete computer simulation
- The process:
 - Build a translator (translates source code to the machine code of an idealized computer)
 - Build a simulator for the idealized computer
- Evaluation of operational semantics:
 - Good if used informally (language manuals, etc.)
 - Extremely complex if used formally (e.g., VDL), it was used for describing semantics of PL/I.

Operational Semantics (continued)

- Uses of operational semantics:
 - Language manuals and textbooks
 - Teaching programming languages
- Two different levels of uses of operational semantics:
 - Natural operational semantics
 - Structural operational semantics
- Evaluation
 - Good if used informally (language manuals, etc.)
 - Extremely complex if used formally (e.g., VDL)

Denotational Semantics

- Based on recursive function theory
- The most abstract semantics description method
- Originally developed by Scott and Strachey (1970)

Denotational Semantics – continued

- The process of building a denotational specification for a language:
 - Define a mathematical object for each language entity
 - Define a function that maps instances of the language entities onto instances of the corresponding mathematical objects
- The meaning of language constructs are defined by only the values of the program's variables

Denotational Semantics: program state

- The state of a program is the values of all its current variables

$$s = \{ \langle i_1, v_1 \rangle, \langle i_2, v_2 \rangle, \dots, \langle i_n, v_n \rangle \}$$

- Let **VARMAP** be a function that, when given a variable name and a state, returns the current value of the variable

$$\text{VARMAP}(i_j, s) = v_j$$

Decimal Numbers

`<dec_num>` → '0' | '1' | '2' | '3' | '4' | '5' |
'6' | '7' | '8' | '9' |
`<dec_num>` ('0' | '1' | '2' | '3' |
'4' | '5' | '6' | '7' |
'8' | '9')

$M_{\text{dec}}('0') = 0, \quad M_{\text{dec}}('1') = 1, \quad \dots, \quad M_{\text{dec}}('9') = 9$

$M_{\text{dec}}(<\text{dec_num}> '0') = 10 * M_{\text{dec}}(<\text{dec_num}>)$

$M_{\text{dec}}(<\text{dec_num}> '1') = 10 * M_{\text{dec}}(<\text{dec_num}>) + 1$

...

$M_{\text{dec}}(<\text{dec_num}> '9') = 10 * M_{\text{dec}}(<\text{dec_num}>) + 9$

Expressions

- Map expressions onto $Z \cup \{\text{error}\}$
- We assume expressions are decimal numbers, variables, or binary expressions having one arithmetic operator and two operands, each of which can be an expression

Expressions

```
Me(<expr>, s) Δ=
  case <expr> of
    <dec_num> => Mdec(<dec_num>, s)
    <var> =>
      if VARMAP(<var>, s) == undef
      then error
      else VARMAP(<var>, s)
    <binary_expr> =>
      if (Me(<binary_expr>.<left_expr>, s) == undef
        OR Me(<binary_expr>.<right_expr>, s) =
          undef)
      then error
      else
        if (<binary_expr>.<operator> == '+' then
          Me(<binary_expr>.<left_expr>, s) +
            Me(<binary_expr>.<right_expr>, s)
        else Me(<binary_expr>.<left_expr>, s) *
          Me(<binary_expr>.<right_expr>, s)
    ...
```


Assignment Statements

- Maps state sets to state sets $\cup \{\text{error}\}$

```
Ma(x := E, s) Δ=
  if Me(E, s) == error
    then error
    else s' =
      {<i1, v1'>, <i2, v2'>, ..., <in, vn'>},
      where for j = 1, 2, ..., n,
        if ij == x
          then vj' = Me(E, s)
          else vj' = VARMAP(ij, s)
```

Logical Pretest Loops

- Maps state sets to state sets $U \{\text{error}\}$

```
M1(while B do L, s) Δ=  
  if Mb(B, s) == undef  
    then error  
  else if Mb(B, s) == false  
    then s  
  else if Ms1(L, s) == error  
    then error  
  else M1(while B do L, Ms1(L, s))
```

Loop Meaning

- The meaning of the loop is the value of the program variables after the statements in the loop have been executed the prescribed number of times, assuming there have been no errors
- In essence, the loop has been converted from iteration to recursion, where the recursive control is mathematically defined by other recursive state mapping functions
 - Recursion, when compared to iteration, is easier to describe with mathematical rigor

Evaluation of Denotational Semantics

- Can be used to prove the correctness of programs
- Provides a rigorous way to think about programs
- Can be an aid to language design
- Has been used in compiler generation systems
- Because of its complexity, it are of little use to language users

Axiomatic Semantics

- Based on formal logic (predicate calculus)
- Original purpose: formal program verification
- Axioms or inference rules are defined for each statement type in the language (to allow transformations of logic expressions into more formal logic expressions)
- The logic expressions are called *assertions*

Axiomatic Semantics (continued)

- An assertion before a statement (a *precondition*) states the relationships and constraints among variables that are true at that point in execution
- An assertion following a statement is a *postcondition*
- A *weakest precondition* is the least restrictive precondition that will guarantee the postcondition

Axiomatic Semantics Form

- Pre-, post form: $\{P\}$ statement $\{Q\}$
- An example
 - $a = b + 1 \quad \{a > 1\}$
 - One possible precondition: $\{b > 10\}$
 - Weakest precondition: $\{b > 0\}$

Program Proof Process

- The postcondition for the entire program is the desired result
 - Work back through the program to the first statement. If the precondition on the first statement is the same as the program specification, the program is correct.

Axiomatic Semantics: Assignment

- An axiom for assignment statements
 $(x = E): \{Q_{x \rightarrow E}\} \ x = E \ \{Q\}$
- The Rule of Consequence:

$$\frac{\{P\} S \{Q\}, P' \Rightarrow P, Q \Rightarrow Q'}{\{P'\} S \{Q'\}}$$

Axiomatic Semantics: Sequences

- An inference rule for sequences of the form
 $S1; S2$

$\{P1\} S1 \{P2\}$

$\{P2\} S2 \{P3\}$

$$\frac{\{P1\} S1 \{P2\}, \{P2\} S2 \{P3\}}{\{P1\} S1; S2 \{P3\}}$$

Axiomatic Semantics: Selection

- An inference rules for selection
 - **if B then S1 else S2**

$$\frac{\{B \text{ and } P\} S1 \{Q\}, \{(\text{not } B) \text{ and } P\} S2 \{Q\}}{\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}}$$

Axiomatic Semantics: Loops

- An inference rule for logical pretest loops

$\{P\} \text{ while } B \text{ do } S \text{ end } \{Q\}$

$$\frac{(I \text{ and } B) S \{I\}}{\{I\} \text{ while } B \text{ do } S \{I \text{ and } (\text{not } B)\}}$$

where I is the loop invariant (the inductive hypothesis)

Axiomatic Semantics: Axioms

- Characteristics of the loop invariant: I must meet the following conditions:
 - $P \Rightarrow I$ -- the loop invariant must be true initially
 - $\{I\} B \{I\}$ -- evaluation of the Boolean must not change the validity of I
 - $\{I \text{ and } B\} S \{I\}$ -- I is not changed by executing the body of the loop
 - $(I \text{ and } (\text{not } B)) \Rightarrow Q$ -- if I is true and B is false, Q is implied
 - The loop terminates -- can be difficult to prove

Loop Invariant

- The loop invariant I is a weakened version of the loop postcondition, and it is also a precondition.
- I must be weak enough to be satisfied prior to the beginning of the loop, but when combined with the loop exit condition, it must be strong enough to force the truth of the postcondition

Evaluation of Axiomatic Semantics

- Developing axioms or inference rules for all of the statements in a language is difficult
- It is a good tool for correctness proofs, and an excellent framework for reasoning about programs, but it is not as useful for language users and compiler writers
- Its usefulness in describing the meaning of a programming language is limited for language users or compiler writers

Denotation Semantics vs Operational Semantics

- In operational semantics, the state changes are defined by coded algorithms
- In denotational semantics, the state changes are defined by rigorous mathematical functions

Summary

- BNF and context-free grammars are equivalent meta-languages
 - Well-suited for describing the syntax of programming languages
- An attribute grammar is a descriptive formalism that can describe both the syntax and the semantics of a language
- Three primary methods of semantics description
 - Operation, axiomatic, denotational